

Orientação a Objetos

João Eduardo Montandon

DCC024

Em POO... nós não dizemos "O computador sabe como mover o ponto", nós dizemos "O ponto sabe como mover a si mesmo". *Isso faz toda a diferença.*

Introdução

- Orientação a Objetos representa um estilo de programar, uma forma de pensar.
 - Esse estilo favorece a **antropomorfização** dos objetos.
 - Um objeto sabe como executar determinadas ações.
 - Um objeto sabe como "conversar" com outros objetos.
-

Programar Orientado a Objetos consiste em mapear as funcionalidades do sistema como ações a serem executadas a partir dos objetos do programa.

Programação Orientada a Objetos

! =

Linguagem Orientada a Objetos

Exemplo: Java

Todo programa em Java é Orientado a Objetos? 🤔

Passo 1: Retire as ações associadas a cada objeto.

```
public class Node {  
    public String data;  
    public Node next;  
}
```

```
public class Stack {  
    public Node top;  
}
```

Passo 2: Implemente as ações fora de qualquer escopo específico

```
public class Main {  
    private static void add(Stack s, String data) {  
        Node n = new Node();  
        n.data = data;  
        n.next = s.top;  
        s.top = n;  
    }  
    private static String remove(Stack s) {  
        String data = s.top.data;  
        s.top = s.top.next;  
        return data;  
    }  
    private static boolean isEmpty(Stack s) {  
        return s.top == null;  
    }  
    ...  
}
```

Passo 3: Profit!

```
public class Main {  
    ...  
    public static void main(String[] args) {  
        Stack s = new Stack();  
        add(s, "O rato");  
        add(s, "roeu");  
        add(s, "a roupa");  
        add(s, "do rei");  
        add(s, "de roma");  
  
        while(!isEmpty(s)) {  
            System.out.println(remove(s));  
        }  
    }  
}
```

Mas e a `MyCollection` ...

Como implementar polimorfismo em uma linguagem que não suporta isso? 🤔

Já viram estratégia similar em outro lugar? 🤔

```
public class MyCollection {
    public static final int STACK = 0;
    public static final int QUEUE = 1;
    public static final int PRIORITYQUEUE = 2;

    public int type;
    public Node head;
    public Node tail; // apenas quando type != stack
    public int length; // apenas quando type != stack
}
```

Como fazer o mesmo método suportar ações distintas?

```
private static void add(MyCollection c, String data) {
    if(c.type == MyCollection.STACK) {
        Node n = new Node();
        n.data = data;
        n.next = s.top;
        s.top = n;
    } else if (c.type == MyCollection.QUEUE) {
        // implementação para fila
    } else if (c.type == MyCollection.PRIORITYQUEUE){
        // implementação para fila de prioridade
    }
}
```

Problemas

- Dados não estão mais encapsulados.
 - Condicionais de tipo espalhadas em vários métodos.
 - Implementação muitas vezes tediosa, e susceptível a erros.
-

Principais Características

1. Classes
 2. Herança
 3. Encapsulamento
 4. Polimorfismo
-

Classes

Desempenham papéis variados.

- Permite agrupar semanticamente dados e métodos.
 - Funciona como um molde para os objetos concretos do programa.
 - Proporciona relacionamento por herança.
 - Servem como tipos.
-

Existe linguagem Orientada a Objetos sem classes? 🤔

Prototypes

```
var pessoa = {  
  nome: "João Eduardo",  
  idade: 34,  
  podeDirigir: function() {  
    return this.idade >= 18;  
  }  
}  
  
var outraPessoa = Object.create(pessoa);
```

- Um *prototype* é um objeto que é copiado para produzir novos objetos.
- A padronização se dá pela réplica desses objetos.

O que perdemos com isso? 🤔

Se não existe uma classe...

- Como trabalhar com tipos? *tipagem dinâmica*

- Como lidar com herança? *delegation*
-

Herança

Relacionamentos são *is-a* ou *has-a*:

- *has-a*, um veículo tem um motor.
- *is-a*, um carro é um veículo.

Herança é o mecanismo que permite a construção de relacionamentos *is-a*.

Ideia é simples: Uma classe *derivada herda informações* de uma classe *base*.

Quais informações seriam essas? 🤔

- Posso herdar características de mais de uma classe diretamente?
 - Java: Não! / C++: Sim
 - Existe uma classe base universal?
 - Java: Sim! / C++: Não
 - É possível modificar os comportamentos herdados? Como?
-

Encapsulamento

- Encontrada em basicamente toda a linguagem moderna.
 - Permite estruturar o programa em pequenos grupos isolados.
 - Acessados por uma interface controlada.
 - Membros internos são invisíveis para fora do grupo.
-

- Java/C++: `private`, `protected`, `public`
 - Python: tudo é público
 - Eiffel: Permite especificar as classes que terão acesso aos membros.
-

Polimorfismo

- Especialmente importante para linguagens OO.
 - Subtipos: O comportamento de um objeto pode ser definido em tempo de execução.
-

```
public static void flashoff(Drawable d, int k) {  
    for (int i = 0; i < k; i++) {  
        d.show(0,0);  
        d.hide();  
    }  
}
```

- `Drawable` é uma interface.
 - Tipo real de `d` é conhecido apenas em tempo de execução (Subtipos).
-

Dynamic Dispatch

```
class Animal {  
    public void eat() { System.out.println(this + " is eating"); }  
    public String toString () { return "Animal"; }  
}  
class Mammal extends Animal {  
    public void suckMilk() { System.out.println(this + " is sucking"); }  
    public String toString () { return "Mammal"; }  
}  
class Dog extends Mammal {  
    public void bark() { System.out.println(this + " is barking"); }  
    public String toString () { return "Dog"; }  
}
```

```
Animal a2 = new Dog();  
a2.eat();
```

O que vai ser impresso? 🤔

```
Animal a = new Dog();
```

- *Dynamic Dispatch* adia a definição do tipo para a execução do programa.
 - Favorece aplicação do polimorfismo.
- Ligação estática: Tipo de `a` é `Animal`.

- Ligação dinâmica: Tipo de `a` é `Dog`.
 - Java sempre usa dynamic dispatch!
-

Duck Typing

If it walks like a duck and it quacks like a duck, then it must be a duck.

-- [Duck typing - Wikipedia](#)

- Definir um objeto a partir do seu comportamento, ao invés do seu contrato.
 - O contrato não impõem restrições para a linguagem.
 - Flexibilidade total!
-

```
var car = {  
  beep: function() {  
    console.log("BEEP!");  
  }  
};  
var bike = {  
  beep: function() {  
    console.log("Prim prim!");  
  }  
};  
var lista = [bike, car];  
lista[0].beep();  
lista[1].beep();
```

Houve polimorfismo aqui? 🤔

O polimorfismo é uma forma que linguagens de tipagem estática tem de alcançar essa flexibilidade, parcialmente!

Considerações Finais

- Orientação a Objetos é um estilo de programação.
- Linguagens adotam estratégias variadas para suportar esse estilo.